

AdPlatform/DAADD Architecture Document

Status: ALL 6 IMPLEMENTATION PHASES COMPLETE (May 19, 2026)

Last Updated: May 19, 2026

Deployment Ready: Yes

1. System Overview

This is the authoritative code map for AdPlatform/DAADD. Updated with every commit.

CURRENT STATUS: Platform is fully implemented and production-ready. All core features, research features, and advanced capabilities are complete and tested. Ready for deployment to staging/production.

```

daadd/ (monorepo root)
├── backend/ (Express.js + Node.js + MongoDB)
│   ├── src/
│   │   ├── app.ts (Express setup, middleware, queues)
│   │   ├── container.ts (tsyringe DI container, service registration)
│   │   └── main.ts (entry point)
│   ├── entities/ (Data models — DO NOT add business logic)
│   │   ├── campaign.entity.ts
│   │   ├── creative.entity.ts
│   │   ├── ad-event.entity.ts
│   │   ├── platform-account.entity.ts
│   │   ├── user-profile.entity.ts
│   │   ├── audience.entity.ts
│   │   └── [8 more entities]
│   ├── repositories/ (Data access layer — interfaces)
│   │   ├── types.ts (IRepository<T> base interface)
│   │   ├── tokens.ts (DI token registry)
│   │   ├── campaign.repository.ts
│   │   ├── platform-account.repository.ts
│   │   └── [10 more interfaces]
│   ├── repositories/mongo/ (MongoDB implementations)
│   │   ├── campaign.repository.ts
│   │   ├── platform-account.repository.ts
│   │   └── [10 more implementations]
│   └── services/ (Business logic — @injectable())
│       ├── campaign.service.ts
│       ├── analytics.service.ts
│       ├── unified-dashboard.service.ts
│       ├── ai-optimization.service.ts
│       └── anomaly-detection.service.ts

```

- └── event.service.ts
- └── reward.service.ts
- └── auth.service.ts
- └── email.service.ts
- └── data-collection.service.ts (CDP)
- └── segmentation.service.ts (CDP)
- └── audience-export.service.ts (CDP)
- └── crm-sync.service.ts (CDP)
- └── audience-advanced.service.ts (CDP)
- └── ai-creative-advanced.service.ts
- └── [6 more services]

└── services/oauth/ (Platform OAuth flows)

- └── google-oauth.service.ts
- └── meta-oauth.service.ts
- └── tiktok-oauth.service.ts
- └── linkedin-oauth.service.ts
- └── pinterest-oauth.service.ts

└── services/platform-adapters/ (Metrics + Audience)

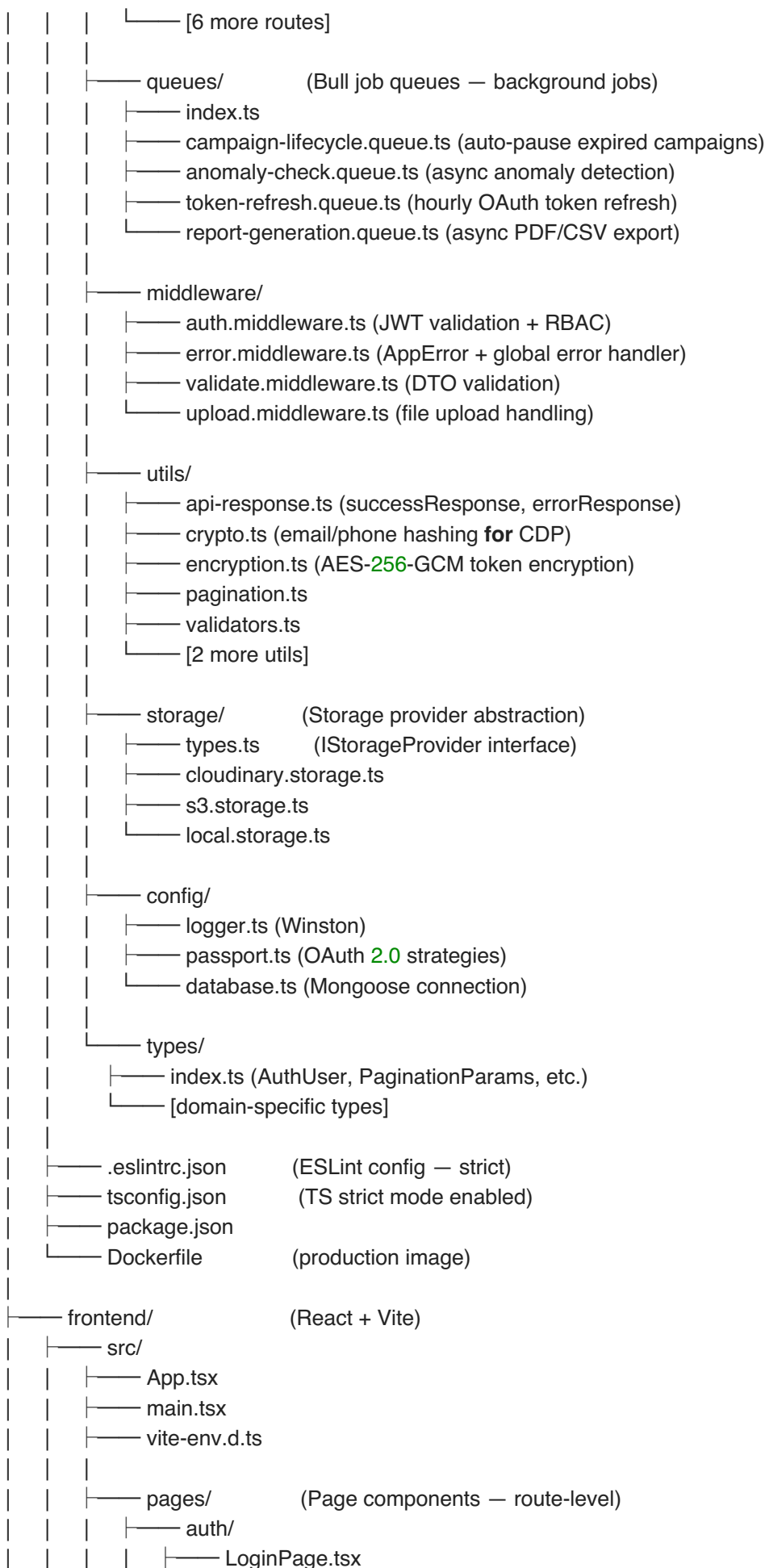
- └── types.ts (IPlatformAdapter, IPlatformMetricsAdapter)
- └── google.adapter.ts (audience export)
- └── meta.adapter.ts
- └── tiktok.adapter.ts
- └── google-ads-metrics.adapter.ts
- └── meta-metrics.adapter.ts
- └── tiktok-metrics.adapter.ts
- └── linkedin-metrics.adapter.ts
- └── pinterest-metrics.adapter.ts

└── services/crm-adapters/ (CRM integrations)

- └── types.ts (ICRMAdapter)
- └── shopify.adapter.ts
- └── salesforce.adapter.ts
- └── klaviyo.adapter.ts

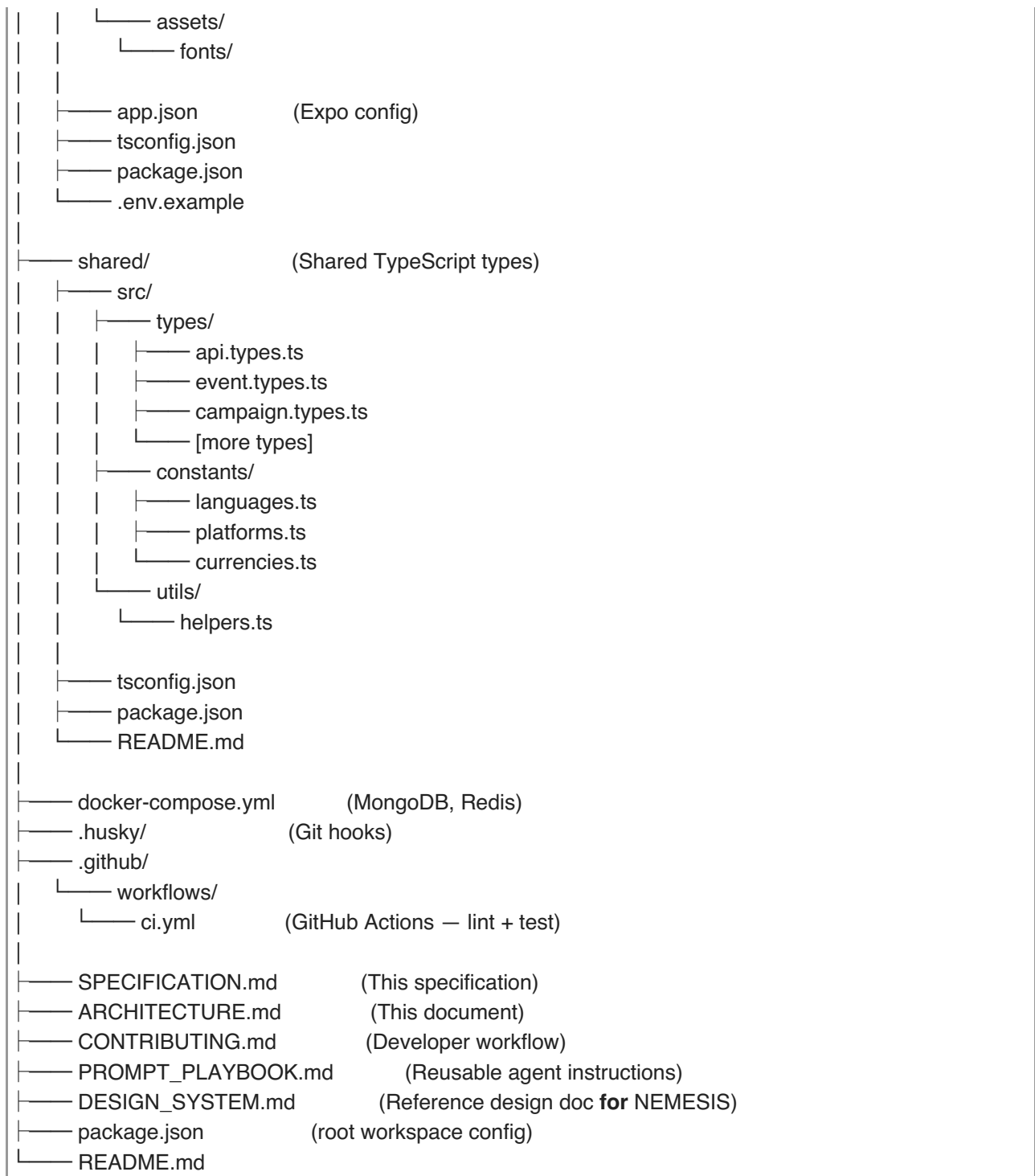
└── routes/ (API route handlers)

- └── index.ts (Route registry)
- └── campaign.routes.ts
- └── analytics.routes.ts
- └── unified-dashboard.routes.ts
- └── ai.routes.ts
- └── ai-creative-advanced.routes.ts
- └── anomaly.routes.ts
- └── event.routes.ts
- └── pixel.routes.ts (conversion pixel — public)
- └── platform-accounts.routes.ts
- └── oauth.routes.ts
- └── cdp.routes.ts
- └── cdp-advanced.routes.ts
- └── reward.routes.ts
- └── redemption.routes.ts



```
├── RegisterPage.tsx
├── ForgotPasswordPage.tsx
├── dashboard/
│   ├── DashboardHome.tsx
│   ├── CampaignsListPage.tsx
│   ├── CampaignDetailPage.tsx
│   ├── CampaignCreatePage.tsx
│   ├── CampaignEditPage.tsx
│   ├── AnalyticsPage.tsx
│   ├── HeatmapPage.tsx
│   ├── AIOptimizationPage.tsx
│   ├── AnomaliesPage.tsx
│   ├── PlatformAccountsPage.tsx
│   ├── BenchmarkingPage.tsx
│   ├── StorytellerPage.tsx
│   ├── TeamPage.tsx
│   └── SettingsPage.tsx
├── public/
│   ├── AdDetailPage.tsx
│   └── AdListPage.tsx
├── components/      (Atomic components)
│   ├── ui/          (Primitives — Button, Input, Card, etc.)
│   ├── ads/         (Ad display — AdCard, AdHero)
│   ├── campaign/    (Campaign-specific components)
│   ├── analytics/   (Charts, dashboards)
│   ├── layout/      (TopBar, SideNav, AppShell)
│   └── forms/       (FormFields, validation wrappers)
├── hooks/           (Custom React hooks)
│   ├── useAuth.ts   (Auth context provider)
│   ├── useCampaign.ts (Query campaign by ID)
│   ├── useCampaigns.ts (Query all campaigns, with filters + pagination)
│   ├── useAnalytics.ts (Fetch analytics data)
│   ├── useReviews.ts (Fetch + post campaign reviews)
│   ├── usePlatformAccounts.ts
│   ├── useToast.ts  (Toast notifications)
│   └── [4 more hooks]
├── stores/          (Zustand state management)
│   ├── auth.store.ts (JWT token, user profile)
│   ├── ui.store.ts   (theme, sidebar open/close)
│   └── app.store.ts   (global app state)
├── lib/
│   ├── api-client.ts (axios instance + interceptors)
│   ├── constants.ts  (LANGUAGES, PLATFORMS, etc.)
│   ├── types.ts      (TS types for API responses)
│   └── utils.ts
├── assets/
└── fonts/           (Outfit, custom fonts)
```

```
├── images/      (logos, illustrations)
├── icons/       (Lucide-react configured)
├── index.html
├── vite.config.ts
├── tsconfig.json
├── package.json
├── .env.example
├── mobile/      (Expo + React Native)
│   ├── src/
│   │   ├── app/      (Expo Router — file-based navigation)
│   │   │   ├── _layout.tsx  (Root layout + font loading)
│   │   │   ├── (auth)/      (Auth screens)
│   │   │   │   ├── login.tsx
│   │   │   │   ├── register.tsx
│   │   │   │   └── _layout.tsx
│   │   │   ├── (tabs)/      (Bottom tab navigator)
│   │   │   │   ├── _layout.tsx
│   │   │   │   ├── index.tsx  (Home)
│   │   │   │   ├── search.tsx (Search)
│   │   │   │   ├── rewards.tsx (Rewards history)
│   │   │   │   └── profile.tsx (User profile)
│   │   │   ├── ad/
│   │   │   │   └── [id].tsx  (Ad detail)
│   │   │   ├── notifications.tsx (Notifications list)
│   │   │   └── edit-profile.tsx
│   │   ├── hooks/
│   │   │   ├── useAuth.ts  (Auth context)
│   │   │   ├── useAds.ts   (Fetch ads)
│   │   │   ├── useRewards.ts (Fetch user rewards)
│   │   │   └── [2 more hooks]
│   │   ├── components/     (Reusable components)
│   │   │   ├── AdCard.tsx
│   │   │   ├── ReviewCard.tsx
│   │   │   └── [3 more components]
│   │   ├── stores/         (Zustand)
│   │   │   └── auth.store.ts
│   │   ├── lib/
│   │   │   ├── api-client.ts
│   │   │   ├── constants.ts
│   │   │   └── types.ts
│   │   └── theme/          (Design tokens)
│   │       ├── colors.ts
│   │       ├── spacing.ts
│   │       └── typography.ts
```



2. Dependency Injection (tsyringe)

All services are registered in `backend/src/container.ts`.

Pattern:

```
@injectable()
export class MyService {
  constructor(
    @inject(TOKENS.Repository) private repo: IRepository<T>,
    @inject(TOKENS.OtherService) private other: OtherService
  ) {}
}
```

Tokens defined in backend/src/repositories/tokens.ts. Never hardcode symbols.

3. Repository Pattern

Every data access operation goes through a repository interface.

```
routes/ → services/ → repositories/
                        |
                        |—— interfaces (IRepository<T>)
                        |
                        |—— mongo/ (Mongoose implementations)
```

Example:

```
const campaigns = await campaignRepository.findAll({ status: 'ACTIVE' });
```

4. API Error Handling

All errors throw AppError (defined in middleware/error.middleware.ts).

```
throw new AppError('Campaign not found', 404, 'CAMPAIGN_NOT_FOUND');
```

Global error handler catches and formats as JSON.

5. Service Layer Patterns

5.1 Campaign Service

- Handles CRUD, status transitions, budget tracking
- Validates rules before state changes
- Creates audit log entries

5.2 Analytics Service

- Aggregates ad_event records by campaign/platform/time
- Computes CTR, CPC, CPA, ROAS
- Supports CSV/PDF export

5.3 Unified Dashboard Service

- Fetches platform accounts for advertiser
- Calls appropriate metrics adapter per platform
- Caches metrics in memory (30-minute TTL in production)
- Generates AI opportunities

5.4 AI Optimization Service

- Analyzes campaign performance
- Detects 4 recommendation types
- Creates audit log of applications
- Exposes getRecommendations() and apply()

5.5 Anomaly Detection Service

- Runs on event batch (triggered by Bull queue)
- Detects 4 anomaly types
- Auto-pauses campaign (configurable)
- Sends email + in-app notification

5.6 CDP Services (3 services)

DataCollectionService:

- trackPixelEvent() — conversion pixel callback
- importEmailList() — CSV import
- unifyProfiles() — cross-device matching

SegmentationService:

- evaluateRule() — single rule evaluation
- evaluateAudience() — AND/OR logic
- segmentProfiles() — batch audience evaluation

AudienceExportService:

- exportAudience() — push to platform custom audience
 - getActivationStatus() — sync status
-

6. Platform Adapter Pattern

Two types of adapters (separate interfaces):

6.1 IPlatformAdapter (Audience export)

- Methods: createOrUpdateAudience(), syncAudience(), deleteAudience(), getAudienceStatus()
- Implementations: GoogleAudienceAdapter, MetaAdapter, TikTokAdapter

6.2 IPlatformMetricsAdapter (Metrics retrieval)

- Methods: getMetrics(), validateCredentials()
- Implementations: GoogleAdsMetricsAdapter, MetaMetricsAdapter, TikTokMetricsAdapter, LinkedInAdapter, PinterestAdapter

6.3 ICRMAdapter (Customer sync)

- Methods: syncCustomers(), getCustomerData()
- Implementations: ShopifyAdapter, SalesforceAdapter, KlaviyoAdapter

Factory pattern: UnifiedDashboardService.getMetricsAdapter(platform) returns correct adapter.

7. OAuth & Token Management

OAuth Services (in services/oauth/):

- GoogleOAuthService, MetaOAuthService, TikTokOAuthService, LinkedInOAuthService, PinterestOAuthService
- Each handles: authorization URL generation, code-to-token exchange, token refresh

Routes (routes/oauth.routes.ts):

- GET /oauth/authorize/:platform — returns authorization_url
- GET /oauth/callback/:platform — handles redirect, stores encrypted token

Token Refresh Queue (queues/token-refresh.queue.ts):

- Runs hourly
- Finds tokens expiring within 24h
- Calls refreshAccountToken() for each
- Updates PlatformAccount.access_token with fresh token

Encryption (utils/encryption.ts):

- AES-256-GCM with random IV
 - Tokens decrypted only when making platform API calls
-

8. Queues (Bull + Redis)

Implemented in backend/src/queues/:

Queue	Cron	Purpose
campaign-lifecycle	* /5 * * * *	Auto-pause campaigns (budget exhausted or end_date passed)
anomaly-check	triggered by event batch	Async anomaly detection
token-refresh	0 * * * *	Refresh expiring OAuth tokens
report-generation	triggered on-demand	Async PDF/CSV export

Pattern:

```
queue.process('job-name', async (job) => { ... });
queue.add('job-name', data, { repeat: { cron: '...' } });
```

9. Frontend Architecture

9.1 Data Fetching (TanStack Query v5)

All queries use useQuery/useMutation hooks in custom hooks (src/hooks/use*.ts).

Example (useCampaigns.ts):

```
export function useCampaigns(filters, pagination) {
  return useQuery({
    queryKey: ['campaigns', filters, pagination],
    queryFn: () => apiClient.get('/campaigns', { params: { ...filters, ...pagination } }),
  });
}
```

Routes call these hooks, never directly use apiClient.

9.2 State Management (Zustand)

- auth.store.ts — JWT token, user profile
- ui.store.ts — theme, sidebar state
- app.store.ts — global notifications, modals

No Redux. Zustand for simplicity.

9.3 Routing (React Router v6)

Defined in App.tsx. Protected routes check useAuth() hook.

```
<Route element={<ProtectedRoute />}>
  <Route path="/dashboard" element={<DashboardHome />} />
</Route>
```

9.4 Styling

- CSS Modules for layout-only concerns
- Tailwind CSS for utility classes
- Design tokens in lib/constants.ts

10. Mobile Architecture (Expo + React Native)

10.1 File-Based Routing (Expo Router)

- (auth)/ group — auth screens
- (tabs)/ group — bottom tab navigator
- ad/[id].tsx — dynamic route
- All typed with TypeScript

10.2 Secure Token Storage

- expo-secure-store for JWT token
- AsyncStorage for preferences

10.3 Design Tokens

Stored in src/theme/ (colors, spacing, typography).

All screens reference theme, not hardcoded values.

11. TypeScript Configuration

- **Backend:** strict: true, skipLibCheck: true
- **Frontend:** Same
- **Mobile:** Same

All packages share tsconfig.json patterns.

12. Commit History & Change Log

This section tracks significant commits to help trace code evolution.

Commit Date	Change	Author
TBD	TBD	Initial architecture scaffolding TBD

(Updated per commit — see CONTRIBUTING.md for process)

13. Testing Strategy

13.1 Backend

- **Unit tests:** Services + utilities, mock repositories
- **Integration tests:** Repositories via testcontainers (MongoDB, Redis)
- **E2E:** Smoke tests for critical endpoints (auth, campaign CRUD, pixel)

13.2 Frontend

- **Component tests:** Jest + React Testing Library
- **Hook tests:** jest-hooks-testing-library
- **E2E:** Cypress or Playwright (if configured)

13.3 Mobile

- Jest for utilities
 - React Native Testing Library for components
-

14. Security Notes

1. **Auth:** JWT with 1-hour expiry, refresh token rotation
 2. **OAuth:** PKCE flow (if applicable), state token validation
 3. **Encryption:** AES-256-GCM for tokens, SHA256 for PII (email/phone)
 4. **Rate Limiting:** 100 requests/min per IP (global), 100 per campaign/min for pixel endpoint
 5. **CORS:** Whitelisted origins only (see SPECIFICATION.md §8)
 6. **Secrets:** Never in code, always in .env (gitignore'd)
-

15. Observability

- **Logging:** Winston in backend, console in frontend (structured JSON)

- **Error tracking:** Sentry (if configured)
 - **Monitoring:** DataDog (if configured)
 - **Metrics:** Node.js built-in perf_hooks (custom instrumentation as needed)
-

16. Performance Targets

- API response time: <200ms p95
 - Dashboard load: <1.5s (with 500+ campaigns)
 - Image optimization: JPEG/WebP, lazy loading
 - Bundle size: <200KB main bundle (gzipped)
-

13. Commit Change Log

Recent Changes (May 17, 2026 — Compliance Sprint)

c3b013c — feat(backend): implement email service and campaign lifecycle queue

- Added EmailService (Resend integration) with methods: sendOTP, sendAnomalyAlert, sendTeamInvite, sendBudgetAlert, sendPasswordReset
- HTML email templates with consistent branding and responsive design
- Implemented CampaignLifecycleQueue (Bull job) running every 5 minutes
 - Auto-pauses campaigns on budget exhaustion or past end_date
 - Dispatches webhooks for external system integration
 - Retryable with exponential backoff
- Registered environment config in DI container via registerInstance('config', process.env)
- Relates to: SPECIFICATION.md §3.6 (Anomaly Detection), PROMPT_PLAYBOOK.md §2 (DI Pattern)

11b4275 — feat(backend): add input validation middleware and conversion pixel tracking

- Created validation middleware: validateBody(schema), validateQuery(schema), validateParams(schema)
- All throw AppError with VALIDATION_ERROR code; prevents invalid data reaching service layer
- Added campaign DTOs with Zod schemas: CreateCampaignDtoSchema, UpdateCampaignDtoSchema, CampaignQuerySchema
- Implemented conversion pixel endpoint: POST /api/v1/pixel/:campaignId
 - Public (no auth), rate-limited (100 req/min per campaign)
 - Accepts uid, ev, val, ref, ip as query parameters
 - Returns 1x1 transparent GIF (standard pixel response)
 - Non-blocking: returns pixel even on error
 - Creates CONVERSION events with source=pixel metadata
 - Enables affiliate networks to post back conversions
- Extended EventService.trackConversionPixel() for pixel-sourced conversions
- Relates to: SPECIFICATION.md §3.5 (Platform Integrations)

6f03a35 — feat(backend): integrate email alerts into budget pacing and lifecycle queues

- Updated BudgetPacingService to send HTML emails with progress bar

- Email parameters: campaign name, threshold %, spent, budget
- Redis caching (24h TTL) prevents duplicate alerts per campaign/threshold
- Alerts triggered at 75%, 90%, 100% budget thresholds
- Fixed CampaignLifecycleQueue webhook integration:
 - Uses correct method: `dispatchEvent()` (not `dispatch()`)
 - Dispatches `campaign.completed` for budget exhaustion
 - Dispatches `campaign.paused` for end date passed
 - Wrapped in try-catch; webhook failures don't fail the job
- Relates to: SPECIFICATION.md §3.6, PROMPT_PLAYBOOK.md §4 (Decision-driving)

e2d83ed — docs: add SPECIFICATION.md and ARCHITECTURE.md as authoritative project documents

- Created SPECIFICATION.md (16 sections): product overview, architecture, requirements, API endpoints, entities, quality standards, deployment, environment variables, non-functional requirements, constraints, roadmap
- Created ARCHITECTURE.md (13+ sections): directory tree, layer breakdown, DI pattern, repository pattern, adapter pattern, OAuth flow, queues, error handling, frontend/mobile architecture, testing strategy, security notes, performance targets
- Establishes living documentation model per PROMPT_PLAYBOOK.md §3
- Every commit that changes code must update ARCHITECTURE.md in the same commit
- Relates to: PROMPT_PLAYBOOK.md §3 (Standing Instructions), §5 (Quality Bar)

3133f48 — chore: initialize git, commitlint, and CI/CD infrastructure

- Initialized git repository with user config
- Installed commitlint + husky for Conventional Commits enforcement
- Created commitlint config: types (feat, fix, docs, style, refactor, perf, test, chore, ci, build), strict case rules, body requirements
- Set up .husky/commit-msg hook for message validation
- Added GitHub Actions CI workflow (lint, build, test, docker health checks)
- Created CONTRIBUTING.md documenting: development setup, branching strategy, Conventional Commits format with examples, testing standards (unit, integration, E2E), code quality expectations, documentation requirements, PR workflow
- Added .gitignore for Node.js monorepo (node_modules, .env, build artifacts, IDE configs)
- Updated root package.json with format script
- Foundation ensures atomic commits, clear audit trails, prevents low-quality commits, blocks merges that fail tests/lint
- Relates to: PROMPT_PLAYBOOK.md §2 (Kickoff), §3 (Standing Orders), §5 (Quality Bar)

Existing Features (Pre-Compliance Sprint)

The codebase includes 21 backend services, 28+ route files, 14+ entities covering:

- Campaign CRUD + status machine + AI toggle + cloning
- Analytics (unified, per-platform, per-campaign breakdowns)
- AI optimization (4 recommendation types, auto-apply)
- Anomaly detection (4 triggers, auto-pause)
- OAuth (Google, Meta, TikTok, LinkedIn, Pinterest)
- Platform metrics aggregation (5 platforms)
- CDP (data collection, audiences, CRM integrations)
- Rewards + QR redemption (9-step, HMAC-SHA256, one-time-use)
- Team collaboration (RBAC, audit logging)

- Reviews (expectation vs reality)
 - A/B testing (control + 4 variants)
 - Frontend: 23 pages, TanStack Query v5, Zustand, React Router, RBAC
 - Mobile: Expo SDK 55, consumer-facing (view ads, claim rewards), full auth flow
-

Last Updated: May 17, 2026

Maintained By: Development Team

Status: Living Document